

Projet de Fin de Cycle (PFC) — Documentation Technique Complète

Système d'Alerte Médicale pour Patients Hospitalisés (Hospital Patient Alarm System)

1. Introduction Générale

Le présent projet consiste en la conception et la réalisation d'un **système d'alerte médicale connecté (Hospital Patient Alarm System)**. Ce système permet aux patients hospitalisés d'émettre des alertes d'urgence depuis leur lit vers un tableau de bord surveillé par le personnel soignant (infirmiers/médecins).

1.1 Problématique

Dans les établissements hospitaliers, la réactivité lors d'un appel d'urgence est un facteur critique. Les systèmes traditionnels filaires sont coûteux à installer, difficiles à maintenir et manquent de flexibilité. Les systèmes purement basés sur le cloud peuvent faillir en cas de perte de connexion Internet.

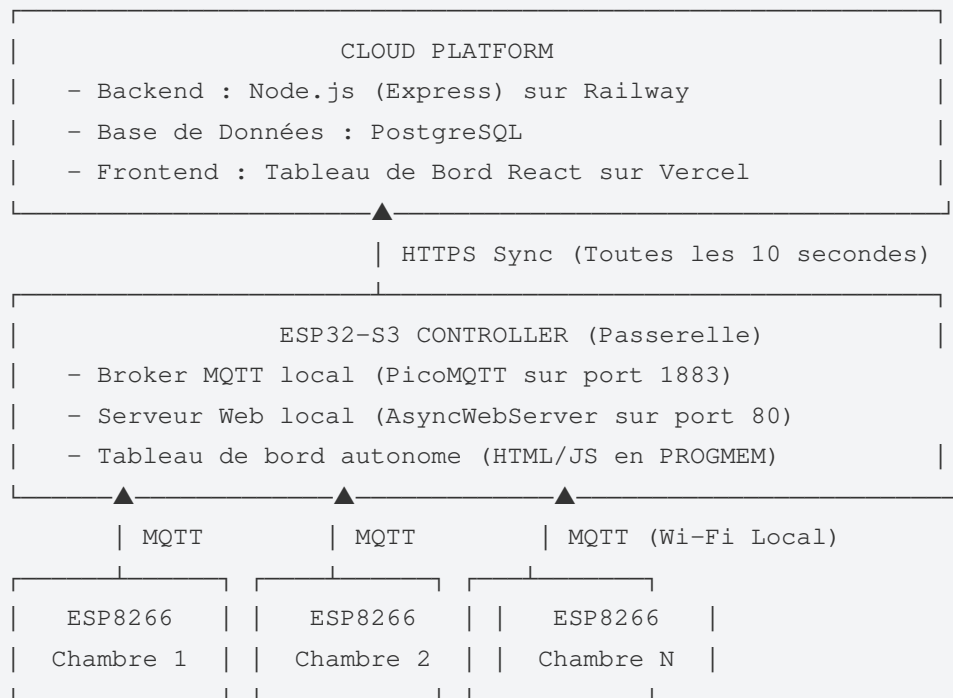
1.2 Solution Proposée

Pour répondre à cette double problématique de fiabilité et de mobilité, nous proposons un système hybride à deux niveaux :

- Un réseau local robuste** basé sur le protocole **MQTT** où un contrôleur central **ESP32-S3** embarque son propre courtier (broker) MQTT et gère en local des terminaux d'appel d'urgence **ESP8266**. Ce réseau fonctionne de manière 100% autonome, même sans Internet.
- Une synchronisation Cloud** bidirectionnelle (Railway + Vercel) qui permet de centraliser les alertes sur une base de données PostgreSQL, de suivre l'état de l'infrastructure à distance, et de gérer les patients via une interface Web d'administration sécurisée ou une application mobile native (Capacitor).

2. Architecture Globale du Système

Le système est segmenté en trois grandes couches : les terminaux clients (Devices), le contrôleur de passerelle local (Controller), et la plateforme de surveillance Cloud (Server + Dashboards).



2.1 Les Terminaux Patients (ESP8266)

Chaque lit de patient est équipé d'un boîtier doté d'un bouton poussoir relié à un module Wi-Fi low-cost **ESP8266**. Ce module communique en Wi-Fi local avec le contrôleur central via le protocole MQTT.

2.2 Le Contrôleur Local (ESP32-S3)

Le cœur logique du réseau local est un **ESP32-S3**. Il cumule plusieurs rôles :

- **Point d'Accès Wi-Fi (SoftAP)** ou client Wi-Fi connecté au réseau de l'hôpital.
- **Broker MQTT local** (via la bibliothèque PicoMQTT) pour orchestrer les terminaux.
- **Serveur Web asynchrone** hébergeant un tableau de bord local (embarqué dans la mémoire flash `PROGMEM`) accessible via IP locale (192.168.4.1 ou DHCP).
- **Générateur d'alerte physique** via un avertisseur sonore (buzzer actif) connecté à une broche GPIO.
- **Passerelle IoT** qui synchronise l'état local avec le serveur Cloud par des requêtes HTTPS périodiques.

2.3 Le Serveur Cloud & Base de Données (Railway)

Un serveur Node.js exécutant Express traite les données de synchronisation envoyées par le contrôleur local. Il persiste l'état du système (liste des périphériques, alertes actives, logs) dans une base de données relationnelle **PostgreSQL** via l'ORM **Drizzle**. Il gère également l'authentification des sessions d'administration.

2.4 Le Dashboard Web & Mobile (Vercel / Capacitor)

Développé en **React 18** avec **Tailwind CSS** et **Radix UI**, ce tableau de bord affiche en temps réel les appels de patients grâce à une connexion **WebSocket** permanente avec le serveur de l'API.

L'application est également encapsulée pour Android via **Capacitor**, permettant l'envoi de notifications locales push et l'usage de retours haptiques (vibrations) sur les téléphones des infirmiers.

3. Spécifications du Matériel (Hardware)

3.1 Schéma des Connexions & Broches (Pin Mapping)

A. Contrôleur Central (ESP32-S3 DevKitC-1)

Le contrôleur n'a besoin que d'un buzzer pour l'alarme sonore physique locale.

Broche ESP32-S3	Composant	Rôle	Description
GPIO 4	Buzzer Actif (+)	Sortie Numérique	Activé (HIGH) lors d'une alerte, désactivé (LOW) sinon.
GND	Buzzer Actif (-)	Masse	Référence 0V
USB-C	Port Série / Alim	Alimentation + Debug	5V / Communication Série à 115200 Bauds

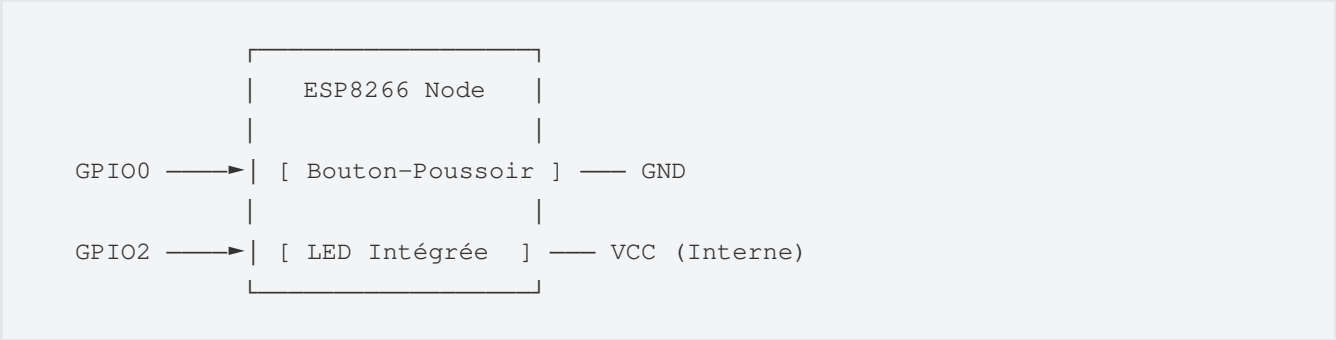


B. Terminal Bouton Patient (ESP8266 NodeMCU)

Chaque boîtier patient possède un bouton-poussoir pour déclencher l'alerte et une LED d'état.

Broche ESP8266	Composant	Mode de Configuration	Description
GPIO 0 (D3)	Bouton Poussoir	INPUT_PULLUP	Connecté au GND lors de l'appui (Actif à l'état BAS).

GPIO 2 (D4)	LED intégrée	OUTPUT	Active à l'état BAS (Active LOW). Indique le statut de connexion et d'alerte.
GND	Bouton Poussoir	Masse	Connecté au pôle opposé du bouton.



3.2 Comportement Lumineux de la LED du Terminal (ESP8266)

La LED du terminal donne un retour visuel instantané sur son état :

- **Clignotement rapide (200ms)** : En cours de connexion au réseau Wi-Fi ou au courtier MQTT.
- **Clignotement lent (500ms)** : Connecté au réseau, mais en attente d'approbation administrative sur le tableau de bord.
- **Allumée Fixe** : Alerte active (le patient a appuyé sur le bouton).
- **Éteinte** : Connecté, approuvé par l'administrateur, système au repos.

4. Modes de Fonctionnement (Operating Modes)

Le système propose 4 modes opératoires sélectionnables lors de la configuration initiale (Setup Wizard) :

Mode	Nom	Description	Wi-Fi Requis	Liaison Cloud
Mode 1	AP Only (Point d'Accès local)	Le contrôleur crée son propre réseau Wi-Fi nommé <code>HospitalAlarm</code> . Tous les terminaux s'y connectent. Interface accessible sur <code>http://192.168.4.1</code> . Idéal pour une cellule isolée sans infrastructure réseau.	Non	Non

Mode 2	STA Only (Station locale)	Le contrôleur se connecte à un réseau Wi-Fi existant (ex: Wi-Fi de l'hôpital). Les terminaux s'y connectent aussi. L'interface locale est servie sur l'adresse IP fournie par le DHCP du réseau local.	Oui	Non
Mode 3	AP + STA Hybrid	Le contrôleur maintient son propre Point d'Accès (HospitalAlarm) tout en se connectant au réseau local de l'hôpital. Les terminaux peuvent se connecter indifféremment sur l'un des deux réseaux.	Oui	Non
Mode 4	Online (Cloud Sync)	Même fonctionnement que le Mode 3, mais le contrôleur synchronise périodiquement l'état local avec le serveur Cloud (Railway) via HTTPS. Les alertes sont acquittables mondialement via le Web.	Oui	Oui (Toutes les 10s)

5. Protocoles de Communication

Le projet exploite quatre protocoles distincts adaptés à chaque niveau de l'architecture :

```
[Terminal ESP8266] <===== MQTT =====> [ESP32 Controller] <===== HTTPS Sync =====>
```

5.1 Protocole MQTT (Réseau Local)

Le protocole MQTT (Message Queuing Telemetry Transport) est utilisé au niveau local sur le port TCP 1883. C'est un protocole léger de type publication/abonnement (Pub/Sub).

Topics MQTT implémentés :

1. `device/{deviceId}/alert` (Device → Controller)

- **QoS** : 1 (Garantit la livraison au moins une fois).
- **Déclencheur** : Appui sur le bouton d'alerte.
- **Charge Utile (Payload)** :

```
{
  "status": "pressed",
  "deviceId": "device-4a2b9f",
  "timestamp": 1716768000
}
```

- **Données Envoyées (Corps de la requête)** : Le contrôleur transmet l'état complet du système local :

```
{
  "uptime": 3600,
  "rssi": -55,
  "mode": 4,
  "wifiError": null,
  "devices": [
    {
      "deviceId": "device-4a2b9f",
      "patientName": "Jean Dupont",
      "bed": "Chambre 4 - Lit A",
      "room": "Cardiologie",
      "approved": true,
      "alertActive": true,
      "online": true,
      "rssi": -65,
      "uptime": 1284,
      "lastSeen": 1716768100,
      "lastUpdatedAt": 1716768100
    }
  ]
}
```

- **Données Reçues (Réponse du serveur)** : Le serveur cloud compare les horodatages (*timestamp-based conflict resolution*). Si une modification a été effectuée côté cloud (ex: un infirmier a cliqué sur "Acquitter l'alerte" depuis l'application Web cloud), le serveur renvoie la commande correspondante dans la réponse HTTP :

```
{
  "commands": [
    {
      "type": "clear_alert",
      "deviceId": "device-4a2b9f"
    }
  ]
}
```

À la réception, l'ESP32 traite la commande en local, coupe le buzzer physique, envoie la trame MQTT `clear_alert` au terminal client concerné, et met à jour son registre en mémoire.

5.3 Protocole WebSocket (Server ↔ Dashboards)

Pour garantir une réactivité instantanée (<100ms) sur les écrans de surveillance du personnel médical,

6.2 Base de Données : Schémas des Tables

Le système utilise PostgreSQL. L'accès s'effectue via Drizzle ORM avec des schémas validés en amont par la bibliothèque **Zod**.

Table 1 : devices

Enregistre la liste des boîtiers d'alerte, les informations des patients affectés et l'état de l'alerte.

Nom de Colonne	Type SQL	Rôle / Valeur par défaut	Description
device_id	TEXT	PRIMARY KEY	Identifiant unique généré à partir de l'adresse MAC du boîtier (ex: device-2b8f41).
patient_name	TEXT	" "	Nom complet du patient occupant le lit.
bed	TEXT	" "	Numéro du lit du patient.
room	TEXT	" "	Numéro de la chambre ou du service médical.
alert_active	BOOLEAN	false	true si le patient a appuyé sur le bouton et que l'alerte n'a pas été acquittée.
last_alert_time	TEXT	NULL	Horodatage ISO du dernier déclenchement d'alarme.
registered	BOOLEAN	false	Indique si le périphérique a contacté le contrôleur au moins une fois.
approved	BOOLEAN	false	Indique si l'administrateur a validé l'appareil et lui a affecté un lit.
online	BOOLEAN	false	Indique si le boîtier émet des pulsations (heartbeats) actives.
last_seen	BIGINT	Unix ms	Timestamp Unix de la dernière activité reçue du terminal.
last_updated_at	BIGINT	Unix ms	Utilisé pour la résolution de conflits lors de la synchro Cloud.

Table 2 : system_settings

Cette table ne contient qu'une seule ligne (Singleton, id = 1) et stocke l'état matériel du contrôleur central ainsi que les commandes en attente d'envoi vers celui-ci.

Nom de Colonne	Type SQL	Valeur par défaut	Description
id	INTEGER	PRIMARY KEY (1)	Clé primaire unique forcée à 1.
controller_last_seen	BIGINT	NULL	Timestamp Unix de la dernière synchronisation réussie de l'ESP32.
controller_uptime	INTEGER	0	Temps de fonctionnement en secondes du contrôleur local.
controller_rssi	INTEGER	0	Force du signal Wi-Fi (en dBm) capté par le contrôleur.
controller_wifi_error	TEXT	NULL	Libellé d'erreur si la connexion Wi-Fi de l'ESP32 échoue.
wifi_mode	INTEGER	1	Mode réseau configuré (1 à 4).
pending_command	TEXT	NULL	Type de commande en attente (ex: clear_alert, REMOVE_DEVICE, CHANGE_MODE).
command_params	TEXT	NULL	Paramètres complémentaires pour la commande (ex: l'ID de l'appareil à supprimer).

7. Sécurité et Résilience du Système

Pour assurer un service critique de type médical, plusieurs mécanismes de protection ont été intégrés :

- 1. Isolation locale en cas de panne Internet :** En mode 4, si la liaison Internet est coupée, l'ESP32-S3 continue d'héberger localement son broker MQTT. Les infirmiers présents dans l'unité de soin peuvent toujours se connecter en Wi-Fi direct sur l'ESP32 (192.168.4.1) pour surveiller les alertes et les acquitter. Une fois la connexion Internet rétablie, la synchronisation

avec le cloud se relance de manière transparente.

2. Détection de perte de liaison (Heartbeat & Timeout) :

- Si un terminal patient (ESP8266) n'envoie pas de pulsation pendant plus de **30 secondes**, le contrôleur local le marque comme hors-ligne (`online = false`) et coupe son voyant sur l'interface, avertissant le personnel d'un dysfonctionnement matériel potentiel.
- Si le contrôleur local (ESP32-S3) n'a pas effectué de synchronisation cloud pendant plus de **12 secondes**, le serveur cloud marque le contrôleur comme hors-ligne, ce qui déclenche un indicateur visuel rouge d'alerte système globale sur l'application Web générale.

3. Sécurité des APIs :

- Les communications Cloud/ESP32 sont protégées par une clé API unique (`X-Device-Key`) passée dans les en-têtes HTTP.
- L'accès au panneau d'administration nécessite une session authentifiée (gérée par Passport.js avec des cookies sécurisés `httpOnly` et configurés en mode `sameSite: "none"` pour le fonctionnement multi-plateforme Vercel-Railway).